

Crusource CRM

Technical Documentation

1. Introduction & Product Overview

1.1 What Is This Platform?

CruSource CRM is a web-based system that helps a sales team manage the entire sales process in one place — from the moment a potential customer is identified, all the way through to closing a deal and keeping a record of every conversation along the way. This document describes the technical design that supports that product: system architecture, technology choices, data model, APIs, and cross-cutting concerns such as security, scalability, and observability.

1.2 Core Modules

- **Leads** — potential customers who haven't been qualified yet.
- **Contacts** — the individual people a rep deals with.
- **Accounts** — the companies or organizations those people belong to.
- **Deals** — active sales opportunities being tracked through a pipeline.
- **Documents** — files attached to accounts or deals.
- **Activities** — tasks, meetings, calls, and notes that make up day-to-day sales work.
- **Reports & Analytics** — visibility into individual, team, and org-wide performance.
- **AI Features** — scoring, drafting, risk alerts, summarization, and a RAG chatbot that reduce manual work.
- **User & Role Management** — controlling who can see and do what.

1.3 Tenancy, Roles & Trial Model

Every organization is a single isolated tenant — there is no data mixing across companies. Users cannot self-register; an Admin provisions every account and assigns one of three roles that governs both login access and data visibility:

Role	Scope
Admin	Sees and manages everything in the organization, including every rep's data and accounts. Only role that can manage users.
Sales Manager	Sees their own records plus every Sales Rep who reports to them ("Reports To" relationship). Can open a rep's dashboard, act on a rep's behalf, assign tasks, and run team-level reports.
Sales Rep	Sees only the records they personally own.

New organizations start on a 14-day free trial with full platform access; after the trial, an active paid subscription is required to continue using the CRM. Subscription state is tracked on the organization record (trial / active / expired / cancelled).

2. System Architecture

CruSource CRM uses a three-tier architecture with an asynchronous AI processing layer, so that LLM calls never block a user-facing request.

2.1 Architecture Layers

Tier	Hosting	Components
Client Tier	Vercel	Next.js App Router (SSR + CSR), TypeScript
API Tier	AWS ECS / Fargate	Application Load Balancer → FastAPI instances ×N → SQS worker tasks ×N
Data Tier	AWS Managed	PostgreSQL 16 + pgvector (Amazon RDS) · ElastiCache Redis 7.x · S3 (documents)
Messaging	AWS SQS	Queues: ai-jobs, notifications, embeddings, each with a dead-letter queue (DLQ)
External	Anthropic/OpenAI	Anthropic/OpenAI API — invoked only from SQS workers, never from the request path

Request & Data Flow

- Client → Load Balancer: the Next.js app calls the API over REST, authenticated with a JWT bearer token.
- Load Balancer → FastAPI: the ALB distributes requests across FastAPI instances.
- FastAPI → Data Tier: application logic reads/writes PostgreSQL, reads/writes the ElastiCache cache, and reads/writes S3 object metadata.
- FastAPI → SQS → Worker: any AI or long-running job is enqueued to SQS rather than processed inline; SQS workers consume the queue and write results back to PostgreSQL.
- SQS → DLQ: messages that exceed MaxReceiveCount are routed to a dead-letter queue for manual inspection/replay.
- Worker → Claude API: LLM calls happen only inside workers.
- Client —→ S3 (dashed): file upload/download bypasses the API entirely via pre-signed URLs.

2.2 Core Principles

Principle	How
API-first	All client ↔ server communication goes through REST.
Stateless backend	JWT-based auth; no server-side sessions.
Async AI	LLM calls are offloaded to SQS workers and never block a request.
Cache-first dashboards	ElastiCache with TTL-based refresh.
Pre-signed uploads	Files go directly to S3, never through the API server.
Row-level security	Every query is filtered by role-based scope at the ORM layer.

3. Technology Stack

The table below is the final selected stack. Section 3.2 gives the rationale, pros, and cons behind each layer; for the three core architecture decisions — Frontend Framework, Backend, and Database — the alternative that was seriously considered is included as well, since these choices carry the largest long-term impact on the project.

3.1 Stack Summary

Layer	Selected Technology & Version
Frontend Framework	Next.js 14+ (App Router) + TypeScript
UI Framework	Tailwind CSS + shadcn/ui
State Management	TanStack Query 5 (server state) + Redux Toolkit 2 (client state)
Data Tables	TanStack Table 8
Drag & Drop	dnd-kit 6
Forms	React Hook Form 7 + Zod 3
Charts	Recharts 2
Backend Framework	FastAPI 0.110+ (Python 3.12+)
Database	PostgreSQL 16 + pgvector
ORM	SQLAlchemy 2 (async) + Alembic 1.13+
Authentication	JWT via python-jose (HS256) + Passlib (Argon2id)
Validation	Pydantic 2
HTTP Client	httpx 0.27+ (async calls to Claude API)
File Storage	AWS S3
Cache	AWS ElastiCache (Redis 7.x)
Background Jobs	AWS SQS (Standard queues)
Frontend Hosting	Vercel
Backend Hosting	AWS ECS / Fargate
Database Hosting	Amazon RDS (PostgreSQL)
Secrets	AWS Secrets Manager
CI/CD	GitHub Actions
Monitoring	AWS CloudWatch

3.2 Rationale, Pros & Cons by Layer

3.2.1 Frontend Framework

Selected: Next.js (App Router) + TypeScript

Chosen for its scalable file-based routing, built-in API routes, and strong deployment support on Vercel — giving the project better long-term structure than a plain single-page app.

Pros	Cons
● Built-in routing and file-based structure ● Strong ecosystem and developer experience ● Better long-term project organization	● Can feel like overkill for simple apps that don't need SEO

Alternative Considered: React (Vite)

A lighter, simpler option considered for its speed, but it needs more manual setup as the project grows.

Pros	Cons
● Very fast local development ● No server-rendering complexity ● Simple for small apps	● Needs manual routing setup ● No built-in API routes ● More upfront setup decisions

3.2.2 UI Framework

Selected: Tailwind CSS + shadcn/ui

Chosen for fast, consistent styling with accessible, customizable components — good for building a polished UI quickly.

Pros	Cons
● Fast styling and iteration ● Highly customizable design system ● Accessible components out of the box	● Verbose class names in markup ● Slight learning curve for new developers

3.2.3 State Management

Selected: TanStack Query + Redux Toolkit

TanStack Query handles server data and caching efficiently, while Redux Toolkit manages complex client-side state predictably.

Pros	Cons
● Strong caching and data syncing ● Predictable, centralized state	● Redux adds some boilerplate ● Can be overkill for simple state needs

3.2.4 Data Tables

Selected: TanStack Table

A headless table library chosen for flexibility and performance, giving full control over the UI.

Pros	Cons
● Highly flexible, full UI control ● Handles large datasets well	● Requires manual UI implementation ● Steeper learning curve

3.2.5 Drag & Drop

Selected: dnd-kit

Chosen for being lightweight, accessible, and easy to customize — used for the Leads and Deals kanban boards.

Pros	Cons
● Lightweight footprint ● Highly accessible ● Fully customizable	● Less plug-and-play than older libraries like react-beautiful-dnd

3.2.6 Forms

Selected: React Hook Form + Zod

Chosen for strong runtime performance and type-safe schema validation, keeping form data reliable across the app.

Pros	Cons
● Fast, minimal re-renders ● Strong type safety with schema validation ● Easy validation setup	● Typing can get complex for reusable form components

3.2.7 Charts

Selected: Recharts

Chosen for being composable and built for React, making it easy to match the product's design system.

Pros	Cons
● Composable, React-native components ● Easy to customize	● Can struggle with very large datasets (SVG rendering)

3.2.8 Backend

Selected: Python + FastAPI

Driven by planned AI integrations, FastAPI was chosen for its mature AI/ML ecosystem, speed, and built-in validation with async support.

Pros	Cons
• Mature, native AI/ML ecosystem • Very fast performance • Built-in request validation (Pydantic)	• Smaller ecosystem for traditional enterprise tooling than Node/Java

Alternative Considered: Node.js + NestJS

Considered to keep the whole stack in one language with strong enterprise architecture, but has a weaker AI/ML ecosystem.

Pros	Cons
• Strong enterprise-grade architecture • Keeps entire stack in one language (TypeScript)	• Weaker AI/ML support • More verbose, decorator-based syntax

3.2.9 Database

Selected: PostgreSQL + pgvector

Chosen for strict relational data handling, referential integrity, and complex joins — while also supporting vector search in the same database.

Pros	Cons
• Strong relational model with referential integrity • Handles complex joins reliably • Native vector search	• Vertical scaling limits • Vector search is slower than dedicated vector databases at large scale

Alternative Considered: MongoDB

Considered for its flexible schema and horizontal scaling, but a poor fit for this project's relational data.

Pros	Cons
• Flexible schema • Fast document retrieval • Easy horizontal scaling	• Poor fit for highly relational data • No strict foreign keys • Complex multi-document transactions

3.2.10 ORM

Selected: SQLAlchemy + Alembic

Chosen for fine-grained SQL control and mature migration tooling via Alembic.

Pros	Cons
• Powerful and mature • Fine-grained SQL control	• Steep learning curve • Can feel heavy for simple queries

3.2.11 Authentication

Selected: JWT (python-jose) + Passlib (Argon2)

Chosen for a stateless, scalable approach that's quick to implement, with Argon2 hashing for strong password security.

Pros	Cons
• Stateless and scalable • Fast to implement	• Harder to instantly revoke tokens without a blocklist

3.2.12 File Storage

Selected: AWS S3

The industry-standard object storage solution, offering high durability at low cost.

Pros	Cons
• Industry standard and highly durable • Cost-effective at scale	• AWS IAM and permissions can be complex to configure

3.2.13 Cache

Selected: AWS ElastiCache

A fully managed caching service with instant high availability and automated backups, removing manual patching and monitoring work.

Pros	Cons
• Extremely fast performance • Fully managed and scalable	• Adds infrastructure cost and architectural complexity

3.2.14 Background Jobs

Selected: AWS SQS

A serverless, highly durable, and scalable managed queue that decouples jobs without relying on a tightly coupled Redis instance.

Pros	Cons
• Fully managed and highly durable • High throughput • Decouples services cleanly	• At-least-once delivery requires idempotent workers

3.2.15 Frontend Hosting

Selected: Vercel

Chosen for zero-config deployments and excellent CI/CD integration, tightly matched to the Next.js framework.

Pros	Cons
• Zero-config deployments • Excellent CI/CD integration	• Vendor lock-in • Can get expensive at scale

3.2.16 Backend Hosting

Selected: AWS ECS / Fargate

Selected for serverless container management that is highly scalable and reliable.

Pros	Cons
• Serverless containers • Highly scalable and reliable	• Complex AWS networking/IAM setup

3.2.17 Database Hosting

Selected: Amazon RDS (PostgreSQL)

Chosen for automated backups and managed scaling, reducing operational overhead for the team.

Pros	Cons
• Automated backups • Managed scaling	• Higher cost compared to self-hosting or a simple VPS

3.2.18 CI/CD

Selected: GitHub Actions

Selected for its tight integration with the code repository and high customizability of pipelines.

Pros	Cons
• Integrated directly with the repository • Highly customizable	• Difficult to debug complex pipelines locally

4. Authentication & Authorization

4.1 Approach

Authentication is stateless and token-based: a user logs in once, receives a signed token that identifies who they are, which organization they belong to, and what role they hold, and presents that token on every

subsequent request. No session state is kept on the server, which lets the API layer scale horizontally without needing sticky sessions.

- The token carries the user's identity, their organization, their role, and an expiry.
- Passwords are never stored in plain text — they are hashed with a strong, modern hashing algorithm designed to resist brute-force attacks.
- Tokens expire on a rolling basis, limiting how long a stolen token remains useful.

4.2 Row-Level Scope

Every piece of data a user can see is filtered automatically, in two layers:

Layer	What It Enforces
Tenant isolation	A user only ever sees data belonging to their own organization — there is no cross-organization data mixing.
Role-based scope	Within an organization, what a user sees depends on their role (see 4.3).

This filtering happens automatically at the data-access layer, so no individual screen or feature has to remember to apply it — it's structurally impossible to bypass from the application code.

4.3 Roles & Access

- **Sales Rep** — only ever sees the records they personally own.
- **Sales Manager** — sees their own records plus the records of every Sales Rep who reports to them; can open a rep's individual dashboard, act on a rep's behalf, assign tasks directly, and generate per-rep performance reports.
- **Admin** — sees and can manage everything in the organization, including every Sales Rep's data and accounts, and is the only role that can manage other users.

The practical effect: the same CRM screen shows different amounts of data depending on who's logged in — a rep gets a focused personal view, while a manager or admin gets progressively broader visibility, without needing separate systems or manual reporting.

User & Role Management (Admin Only)

Found under Settings, this is where an Admin controls system access. An Admin can:

- See every user's email, assigned role, and active/inactive status.
- Add a new user and assign a role immediately.
- Change a user's role as responsibilities change (e.g. promoting a Sales Rep to Sales Manager).
- Deactivate a user to cut off login access immediately, without deleting any records or history they created.
- Assign each Sales Rep to a Sales Manager, which determines what the manager sees as their "team."
- Transfer a user's records — leads, contacts, accounts, deals, activities — to another user when ownership needs to change.

5. Module Data Flows

5.1 Lead Conversion

Converting a qualified lead into a real sales opportunity is the key action in the Leads module — one click creates or links the associated Account and Contact and opens a new Deal. The whole operation happens as a single, all-or-nothing step: either everything is created successfully, or nothing is, so a lead can never end up half-converted.

Built with: Next.js + TypeScript UI with React Hook Form + Zod for the conversion form; TanStack Query for the client-side call; FastAPI + SQLAlchemy on the backend, wrapping the account/contact/deal creation in a single PostgreSQL transaction.

- Before creating a new company record, the system checks whether an account with a matching name already exists, to avoid duplicate accounts piling up.
- If a likely duplicate is found, the rep is warned and can link to the existing account instead of creating a new one.
- If the account is genuinely new, the system creates the account, links a contact to it, and opens a new deal against it, all together.
- The lead itself is marked converted and linked to the new records it produced, so its history stays traceable.

5.2 Deal Pipeline

Deals move through a fixed sequence of stages from creation/conversion to a closed state:

Qualification → Discovery → Proposal → Negotiation → (Closed Won | Closed Lost)

Moving a deal to a new stage automatically refreshes the numbers shown on dashboards, and clears any “at risk” flag on that deal, since a stage change is itself a sign of active progress.

Built with: dnd-kit for the drag-and-drop kanban board, TanStack Query for syncing stage changes to the server and Redux Toolkit for client-side board state, Recharts for the pipeline value visualizations, and a FastAPI endpoint that updates the deal and invalidates the relevant cache entries.

5.3 Activity Timeline

Every task, meeting, call, and note logged against a lead, contact, account, or deal is pulled into one unified, chronological timeline on that record — instead of living in separate tabs. Where a meeting or call has an AI summary attached, the summary is shown by default for quick scanning, with the full original notes always one click away.

Built with: TanStack Table for the timeline list view, shadcn/ui components for the expandable summary/notes toggle, and FastAPI + SQLAlchemy on the backend to assemble the combined, ordered feed.

5.4 Document Handling

Files attached to an account or deal are uploaded and downloaded directly between the user's browser and cloud storage, rather than passing through the application servers. This keeps large file transfers fast and off

the critical path of the rest of the app, while access to every file still follows the same ownership and role-based visibility rules as the record it's attached to.

Built with: Next.js UI for the upload/download controls, AWS S3 for storage, and a FastAPI backend that issues short-lived pre-signed URLs so the browser talks to S3 directly.

6. AI Subsystem

A guiding product principle shapes this entire subsystem: the AI never acts autonomously. It scores, drafts, flags, and summarizes — a person always reviews and decides before anything is sent, dismissed, or acted on. AI output also never creates a new access rule: it inherits the same visibility scope as the record it's attached to.

6.1 Architecture

Feature	How It Runs
Lead Scoring	Rule-based, weighted signals — runs directly against the database, no external AI call.
Deal Risk Alerts	Rule-based, threshold checks — runs on a schedule, no external AI call.
Email Drafting	Offloaded to a background worker, which calls the AI model and stores the result.
Meeting/Call Summarization	Offloaded to a background worker, which calls the AI model and stores the result.
RAG Chatbot	Retrieves the most relevant CRM records for the question, then a background worker calls the AI model with that context.

Anything that calls an external AI model is deliberately offloaded to a background worker rather than handled inline in the request — an AI response can take several seconds to tens of seconds, and the product should never feel like it's hanging while that happens. The user gets an immediate acknowledgment, and the result appears moments later.

6.2 Feature Breakdown

6.2.1 Smart Lead Scoring

Why it matters: *Automatic scoring tells reps which leads to prioritize, instead of guessing.*

- Every open lead is scored 0–100, with a short reason shown alongside it (e.g. “Referral source, contacted within 1 hour”) so a rep understands why, not just a number.
- Scores update automatically overnight, and instantly whenever a lead's status changes or new activity is logged against it.
- The score appears on the Leads list and pipeline board and can be sorted on, so the highest-priority leads rise to the top.
- An Admin can adjust how much weight different signals carry, so scoring reflects what the organization actually considers valuable.

6.2.2 AI-Drafted Follow-up Emails

Why it matters: *Gives reps a fast starting draft for routine follow-ups, with a human still reviewing before it's sent.*

- Available from any Lead, Contact, or Deal with a single click.
- The draft is generated using that record's own history and context, then presented in an editable box — the rep can tweak it, regenerate it, or discard it entirely.
- Nothing is ever sent automatically; the rep always reviews and sends it themselves.

6.2.3 Deal Risk Alerts

Why it matters: *Surfaces at-risk deals before they're lost to neglect.*

- The system automatically flags a deal when it's gone quiet for too long, or its expected close date is approaching without recent progress.
- Each flag carries a short, specific reason (e.g. “No activity in 9 days, Closing Date in 5 days”).
- Flags show up on the Deals list, the pipeline board, and the “Untouched Deals” widget on the Home dashboard, so managers see which specific deals need attention.
- A rep can dismiss a flag as a false alarm, or jump straight into logging an activity to clear it properly.

6.2.4 Meeting & Call Summarization

Why it matters: *Turns raw notes into a quick, scannable summary.*

- A rep types or pastes in notes or a transcript after logging a meeting or call.
- The system generates a short summary and a suggested next step automatically.
- On the shared Activity Timeline, the AI summary is shown by default for scannability; the full original notes are always one click away.
- The suggested next step can be turned into a follow-up task with a single click.

6.2.5 Conversational AI Assistant (RAG Chatbot)

Why it matters: *Lets reps and managers get answers instantly, without digging through screens.*

- A chat panel, available anywhere in the CRM, lets reps and managers ask questions in plain language.
- The assistant answers using the org's own CRM data — deals, activities, contacts, documents — not generic outside knowledge.
- Answers stay scoped to what that person can already see (the same Own/Team/Full rules as the rest of the CRM), and every answer links back to its source record.

Planned Future Integrations

- Daily task reminders sent proactively through the chat, instead of a rep having to go looking.
- Summaries of requested data, so a rep can ask for a quick rundown instead of pulling it together manually.
- End-of-day wrap-up: a rep tells the assistant their completed tasks and tomorrow's plan in chat, and the AI automatically builds tomorrow's Workqueue from it.

7. Caching Strategy

Dashboards and pipeline boards aggregate across many records, which is expensive to recompute on every page load. Results for these views are cached for a short window and served instantly on repeat visits, while any write to the underlying data immediately invalidates the relevant cached view — so the numbers a user sees are never more than a few minutes stale, and never stale at all right after they themselves make a change.

What's Cached	Refresh Behavior
Home dashboard	Refreshes automatically every few minutes, or immediately after a relevant change
Org-wide analytics dashboard	Refreshes automatically every few minutes
Leads / Deals kanban boards	Refreshes immediately after a status or stage change
Pre-built reports	Refreshes periodically

8. File Storage

Documents attached to Accounts or Deals are stored in cloud object storage rather than the database, which is the right fit for potentially large binary files. Files are uploaded once and linked to the record they belong to, keeping paperwork accessible to the whole team rather than scattered across email or shared drives.

- Files are encrypted at rest.
- Access to a file always follows the same ownership and role-based visibility rules as the record it's attached to — there is no separate, looser permission model for documents.
- A reasonable per-file size limit keeps uploads fast and predictable.

Per-file version history is not part of the initial release; it's noted as a future enhancement once the core document workflow is validated.

9. Background Processing

Work that doesn't need to happen instantly — generating an AI draft, summarizing a call, sending a notification, scanning for at-risk deals — is handled by background workers rather than inline in a user's request, so the app stays responsive no matter how long that work takes.

Category	Handles
AI processing	Email drafts, meeting/call summaries, batch lead scoring, deal risk scans
Notifications	Generating and delivering in-app notifications
Search indexing	Keeping the AI assistant's knowledge of CRM records up to date

Each category of work is isolated from the others, so a slow-down or failure in one (say, AI processing) doesn't back up or affect the others. Work that fails repeatedly is set aside for review rather than silently dropped or retried forever.

10. Search & Notifications

10.1 Global Search

A single search bar lets a user find a lead, contact, account, or deal by name, company, or email without needing to know which module it lives in — search results respect the same role-based visibility as everywhere else in the CRM, so a user only ever sees matches they're already allowed to see.

10.2 Notifications

Users are notified in-app when something needs their attention:

- A task is assigned to them.
- A task they own comes due.
- A deal they own is flagged as at risk.

Real-time push notifications are a planned enhancement beyond the initial release.

11. Security

Concern	How It's Addressed
Credential safety	Passwords are never stored in plain text; login tokens are short-lived; secrets are kept in a managed secrets store, not in code.
Data tampering / injection	All data access goes through a query layer that separates code from user input, closing off injection-style attacks.
Cross-site scripting	User-supplied content is never rendered as raw, executable markup.
Broken access control	Role-based scope is enforced automatically on every data access, not left to individual features to implement correctly.
Tenant data leakage	Organization isolation is enforced automatically on every data access.
File upload abuse	Uploads are restricted by file size and type.
Abuse / brute force	Sensitive actions like login are rate-limited per user.

12. Scalability & Performance

12.1 Scaling Approach

Each layer of the system scales independently, based on its own load, rather than the whole application scaling as one unit:

- **Application servers** scale horizontally — more instances are added automatically as request volume grows.
- **Background workers** scale horizontally based on how much queued work is waiting.
- **Database** handles heavy read traffic (reports, dashboards) separately from write traffic, so reporting load never slows down day-to-day data entry.
- **Cache** absorbs repeated reads of the same aggregated data (dashboards, boards) so they don't hit the database on every load.

12.2 Performance Targets

Experience	Target
Everyday page and record loads	Feel instant (sub-second)
Dashboards (cached / freshly computed)	Under 2 seconds / under 5 seconds
Global search	Under half a second
AI email draft	Ready within about 30 seconds
AI chat response	Ready within about 10 seconds
Bulk import of ~1,000 records	Completes within about 30 seconds

13. Observability

The system is built to make problems visible before they become outages, and diagnosable quickly when they happen.

13.1 Monitoring

- Every request is logged with enough context (who, what, when, how long, success or failure) to reconstruct what happened without exposing sensitive data like passwords or personal details in logs.
- The system continuously reports its own health — whether the database, cache, and background workers are reachable and responding.

13.2 Alerting

The team is automatically alerted when key signals drift outside healthy ranges, including:

- An unusual rate of failed requests
- Slower-than-normal response times

- Servers running consistently near capacity
- The database approaching its connection limit
- Background work repeatedly failing and piling up
- The cache no longer absorbing most repeat reads

Appendix A — Key Technical Decisions

A quick-reference summary of the major architectural decisions made for this project and why, beyond the full rationale already covered in Section 3 (Technology Stack).

Decision	Approach Taken	Why
AI processing	Runs in the background, never inline in a request	AI responses can take seconds to tens of seconds — the app should never feel like it's hanging
File uploads/downloads	Go directly between the browser and cloud storage	Keeps large file transfers off the application servers
Lead scoring & deal risk alerts	Rule-based, not AI-generated	Deterministic, fast, and tunable by an Admin, without ongoing AI cost
Access control	Enforced automatically at the data layer, not per-feature	Impossible to accidentally bypass from any individual screen
Caching	Short-lived, invalidated on write	Dashboards stay fast without ever showing meaningfully stale data
Notifications (initial release)	In-app, checked periodically	Simpler to build and operate; real-time push is a planned upgrade